User Guide

Русский · Overview · OKTMO · Reference

Contents

- 1. Launching
- 2. The Window and the Workspace Folders
- 3. How an Operation Card Works
- 4. Operations
- 5. Maintenance
- 6. Working Order

The program is driven from a window: operations are chosen with the mouse, columns are given as Excel letters, and results go into a separate folder. Ordinary work requires no editing of configuration files. The same operations also run without the window, with the console command (see the Reference).

1. Launching

`Start.exe` in the project root opens the program window. If the window is already open, a second start brings it to the front instead of opening another. On the first start the window is Russian; the **RU** and **EN** buttons at the right of the header switch the language on the fly, and the choice is remembered.

If no window appears, or the journal says there is no project Python, see Install.

2. The Window and the Workspace Folders

The window has three parts.

The header. The name, the version and the badges of the machine: the date of the OKTMO registry snapshot and whether Excel and Word are present (a green dot means they are; Excel is needed to move columns and compare tables, Word to convert old `.doc` files). On the right: **SERVICE** (open the program's folders, clear `input` and `output`), **Aa** (font size: normal, larger, smaller), **RU / EN**, the theme switch and **ABOUT** (version, folders, author, licence).

The sections. Under the header there are five big buttons: SOURCES, COLLECT, REFERENCE, MATCH, OKTMO. When a section has several operations, a second row under them holds the tabs of the operations.

Two columns. On the left is the card of the chosen operation. On the right are the working area (SOURCE and TARGET), the RESULT and the JOURNAL. At the bottom is the status line: while an operation runs it shows the course, the time and the **STOP** button.

Workspace Folders

The project works with six managed folders:

Folder	Contents
`input`	Source files
`output`	Results
`report`	JSON run reports
`logs`	Operation logs
`workspace`	Intermediate files
`release`	Result archives

SOURCE is where operations take tables and documents from: a folder or a single file. **TARGET** is where they put the result. At start these are always the project's `input` and `output`: the earlier choice is not remembered. A path can be chosen with the folder button (the source also has a file button), typed into the line, picked from the history, or a file or a folder can be dropped straight onto the window. A path can be pinned — pinned ones come first in the list — and opened in the file manager. The history of paths is kept in `config/path_history.json`. While an operation runs the paths do not change.

The buttons under the paths:

Button	Action
RESET	Return both paths to `input` and `output`; pins stay
LIST	Name the files of the source in the journal: alphabetically, numbered
CHECK	The "Validate input" operation: list the supported spreadsheets
CLEAR	Delete the contents of the source and the target; the folders themselves stay. The window asks and names both paths; what is deleted goes to the Recycle Bin

Spaces and Cyrillic in paths are supported.

Check the list before starting. A command works with the prepared set of files, not with the last one you opened in Explorer.

---

3. How an Operation Card Works

Every operation opens the same way.

- At the top: the name, a line saying what it is for and, on the right, the filled **RUN** button (Ctrl+Enter). The explanations of fields and buttons are in tooltips: they show when the pointer rests on a thing.
- The **fields** are grouped in frames by meaning: Files, Sheets, Columns, Rows, Options, Mode. The frames stand in one, two or three columns by the width of the window. A short field has its name on the left; a choice of several variants and sets of flags are named above. A small orange dot beside a field means its value differs from the default.
- **Advanced** — fields rarely needed: data folder paths, manual overrides.
- The **JOURNAL** on the right is the live course of the operation; a long stage shows one live line instead of a hundred identical ones. The LOGS, REPORT and CONFIG tabs open folders; the buttons on the right copy the journal, clear the panel (the files in `logs` stay) and expand the journal over the whole column.
- The **RESULT** is the outcome of the last operation: what it found and how much. Buttons: **REPORT** (a summary of the report), **FOLDER** (where the workbooks went), **TO SOURCE** (make that folder the source of the next operation); below are **REMOVE EMPTY ROWS** and **PACKAGE OUTPUT AS ZIP** (see Maintenance).

One operation runs at a time. While one runs, another will not start. **STOP** asks the operation to end at the nearest safe point: the workbooks already done stay, and the operation's own Excel and Word are closed. The window may be closed during a run — it asks, and closes once the operation has stopped.

Operations that change the managed workspace ask for confirmation and show what exactly will be affected.

How Columns Are Given

As Excel letters: `B`, `C,D`, `AC`. Ordinal numbers are accepted too. Several columns are separated by commas.

An empty field almost always means automatic mode, not an error.

Where a "header" field exists, the column can be chosen by its name from the first row of the workbook. An explicitly given letter takes priority over the header.

---

4. Operations

Section	Tabs	Described in
SOURCES	CONVERT DOC/XLS · CHECK INPUT · PARSER DIAGNOSTIC	4.8, 4.9
COLLECT	—	4.2
REFERENCE	NORMALIZE · PROCESS	4.4, 4.5
MATCH	ALIGN · TWO TABLES · MOVE COLUMNS	4.3, 4.6, 4.7
OKTMO	KEYS & AREA · UPDATE ROSSTAT	4.1

4.1. OKTMO / Project Keys

The territory registry, search keys, and pins. The registry **is downloaded by the UPDATE ROSSTAT operation** and refreshed by the same one; the cache is rebuilt immediately.

This has its own document: OKTMO and project keys.

4.2. Address Collection

Recursively collects address-like values from XLSX, DOCX, ODT, TXT, CSV, MD, and searchable PDF, merges slot evidence, drops incomplete rows, and writes an address workbook.

Use it when you need an inventory first and matching later.

Action

- *New table* — collect everything from scratch.
- *Append to ready* — add collected addresses to an existing workbook.

Sources. "Source folder" is an optional folder for recursive scanning; empty means the whole source folder. "Address table folder" is where the ready address workbook lives.

Columns. "Address columns" restrict parsing to specific XLSX columns; empty means all cells and all supported files are scanned. For append mode, set "Collected address column" and "Target address column".

Slots. One field — the **Address ruler**. At the top it shows how the address will be written (an example; it cannot be edited). Under the example there is a chip for each slot of the hierarchy: its tick switches the slot on, the number at the right is its place among the switched-on ones, and the chip itself can be dragged to change the order. Below are the "Technical keys" (the OKTMO code, the base of a house and others): they only have ticks. The arrow button returns the enabled slots and the order to the defaults of the manifest. The full slot list is in the Reference.

Options

Option	What it does
Skip microdistrict when street exists	The microdistrict column stays, but is excluded from the generated line
Clean new workbook	A compact workbook with the address column only, instead of the full one
Merge duplicates	Merge candidates with the same address key and vote for the best slot values
Sanitize incomplete	Drop street without house, house without street, subject-only addresses, and table dumps
Write rejected sheet	Rejected candidates go to a separate sheet for audit
Enable OKTMO keys	Resolve territory from the registry and the project keys

Benchmark workbook. An optional workbook with a control address column. It is used only to score collection quality and writes `report/address_collection_benchmark.json`.

Result. An `AddressCollection_*.xlsx` workbook in the result folder and `report/address_collection_summary.json`.

After collecting, check for: empty values, duplicates, cells holding several addresses, and rows that name an organisation but no address.

4.3. Address Alignment

The main mode. Compares address columns against a reference column, rearranges rows so that addresses line up with their pairs, and moves linked columns along with them.

Columns. "GT column" is the reference address column (empty = auto-detect). "Address columns" is what to align; empty means take up to five columns to the right of the reference.

Alignment mode

- *Fast alignment* — matching with no extra processing.
- *Alignment + normalization* — the same search, then a cleanup of the aligned address columns in the result.

Linked columns. Values from linked columns travel together with the matched address, so the data does not drift apart.

Mode	Behaviour
Auto	The legacy automatic behaviour
None	Pure address-column alignment
Left of each address	Columns to the left of each address
Right of each address	Columns to the right of each address
Between GT and address	One distant column carries the whole block between the reference and itself

The last mode answers a common case: the reference in `A`, the address in `AC`, and all of `B:AB` has to move with `AC`.

The "Satellite columns" field sets links explicitly when automatic detection does not fit.

Alignment and normalization. "Normalize before match" is a slower mode using the common normalizer. "Collect before match" builds clean candidates from the selected columns before searching; "Whole document" widens collection to the entire workbook. "Default city" names the city removed from component keys during normalized matching.

Result. A result workbook in the output folder and `report/address_alignment_summary.json`. The log carries a per-column summary: how many rows each pass matched, how many were precise, how many relaxed, and how many stayed without a pair.

How to read the summary. A high share of relaxed hits is a reason to look with your eyes. An exact match is evidence; an approximate one is a hypothesis.

4.4. Reference Normalization

Builds an address reference directory from sources, or brings an existing one to the current slot set.

Action

- *Generate* — a new directory from the sources.
- *Update* — extend an existing one.
- *Clean slots* — rebuild obsolete slot columns.

Options. "Merge old duplicates" merges records with the same slot key. "Rebuild old slots" drops obsolete slot columns and writes only the current set. "Clean new workbook" produces a compact result.

Result. An `AddressReference_*.xlsx` workbook and `report/reference_normalization_summary.json`.

Normalization is deterministic: whitespace, punctuation, case, abbreviations, and component extraction are handled the same way from run to run. The original address value is preserved in the result.

The normalized key helps the search but does not replace the evidential source string.

4.5. Reference Processing

Sorts the reference address column by slots, or decomposes it into separate slot columns.

Action: *Sort*, *Deconstruct*, *Sort + deconstruct*.

Slot order here defines the sorting hierarchy: which slot dominates first. Region → municipality → settlement → street → building, or any other order.

Slot column headers: *Full* (`Postal_Index`, `Municipality`) or *Short* (`Index`, `Municipality`).

Remove source address column — during deconstruction the slot columns are inserted and the original is dropped.

Result. The processed workbook and `report/reference_processing_summary.json`.

4.6. Compare Two Tables

Takes any two workbooks, assembles an address from the given columns of each, normalizes, matches buildings, and creates a **separate** comparison workbook. Both source workbooks stay untouched.

Fields. Table A and Table B, their sheets, the address columns of each, and the first data row of each. B's address columns may sit in different positions than A's.

Sort addresses within groups. Off — matched rows come first in A order, then "only A" and "only B". On — rows are sorted by address inside each group.

Result. A comparison workbook with a status column and colour fill:

Status	Colour	Meaning
matched	green	Exactly one pair
matched: N rows of B	yellow	Several candidates, needs review
only A	A tint	No pair
only B	B tint	No pair

Plus `report/table_comparison_summary.json`.

4.7. Move Columns by Address

Finds identical buildings in two workbooks and moves selected columns from the source into the corresponding rows of the primary workbook.

Copying rows will not do here: the row order of the primary workbook, its formulas, formatting, and sheets must stay in place. So the transfer runs through Excel, and the primary workbook is first copied into the result folder — the copy is what gets edited.

Fields. The primary and source workbooks, sheets (several may be checked in the source), the address columns of both, "Fields to append" — what exactly to carry over, the first data rows, and the output header row.

Result. A copy of the primary workbook with the added columns, a status column, and an `AUDIT_ADDRESS_JOIN` audit sheet. Statuses: matched, matched with a count, not found. The report is `report/safe_table_join_summary.json`.

Check that one address did not resolve into several incompatible rows: that is exactly what the status with a count shows.

4.8. Convert DOC/XLS

Turns legacy `.doc` and `.xls` in the sources into `.docx` and `.xlsx`.

Overwrite existing — when off, existing `.docx` / `.xlsx` files are preserved. Same-name PDF/TXT/CSV/MD duplicates left over from earlier conversions are removed as well.

The report is `report/legacy_office_conversion_report.json`.

4.9. Diagnostics

Two sub-commands.

Run parser diagnostic extracts parsed address components from the sources and shows them as they are. This is how to understand why a particular address found no pair: you see what the program took for a street, what for a building, and what for a territory.

Validate input lists the supported spreadsheets in the source folder and clears Office temporary files along the way.

---

5. Maintenance

Button	What it does
CLEAR (working area)	Delete the contents of the chosen source and target; the folders remain, what is deleted goes to the Recycle Bin
SERVICE → "Clear the input and output of the program (an operation)"	An operation: delete the contents of the project's managed `input` and `output`; the folders remain
REMOVE EMPTY ROWS (result panel)	Create copies of results without rows that are empty across all columns
PACKAGE OUTPUT AS ZIP (result panel)	Build `release/address_aligner_output.zip` from the result folder

Empty-row removal only drops rows that are empty entirely. A row with data in a single column counts as non-empty and stays.

Folder cleanup is confined to the managed workspace: `cleanup_managed_workspace_only` and `never_delete_original_inputs` in `config/project.yaml` keep it from reaching outside the project or deleting originals.

6. Working Order

A typical run:

1. **Prepare the OKTMO registry** if you work with territorial context — once with the UPDATE ROSSTAT operation, then again as new snapshots are published.
2. **Collect the keys** of the region or municipality in the OKTMO section.
3. **Place the sources**, check the list.
4. **Run on a small controlled set** — a dozen rows whose answer you know in advance.
5. **Run the full pass.**
6. **Read the report.** A run is not finished until the report has been reviewed.
7. **Open the result** in a spreadsheet application and verify headers, row order, formulas, and carried-over values.

What to Look at in the Report

- empty addresses;
- duplicate keys;
- one-to-many and many-to-one links;
- building and block contradictions;
- low-confidence candidates;
- rows that did not reach the result;
- unexpected changes in row count.

Rules for Safe Work

Preserve the original row order when it matters to the systems that will consume the result.

Do not hand-normalize part of a set before an automatic comparison: half the rows brought to one form and half raw gives a worse result than an entirely raw set.

Keep the sources and the report until the result is accepted.

After changing matching logic, run a small controlled set first.

On failure, keep the log and the report. Fix the source, the reference, or the column mapping — and repeat the run only after you understand the cause.